

Bengali ASR: six models, one GPU

1322 FLEURS bn_in utterances (test + validation) · NVIDIA RTX A5000 · greedy decoding, no external LM

Model	WER	95% CI (clustered)	CER	ms/clip	× real time	Params
Conformer Large	14.93%	[13.81–16.06]	4.55%	77	172×	121.5M
► Qwen3-ASR + Bengali adapter	16.03%	[15.10–16.99]	4.47%	9263	21048 / 38M trained	
ehzawad FastConformer	19.50%	[18.49–20.49]	5.68%	77	174×	115.6M
Whisper Medium	27.89%	[26.76–29.03]	8.95%	1804	7×	763.9M
Wav2Vec2	31.59%	[30.48–32.70]	9.79%	64	209×	315.5M
hishab FastConformer	36.18%	[34.10–38.29]	9.03%	76	174×	115.6M

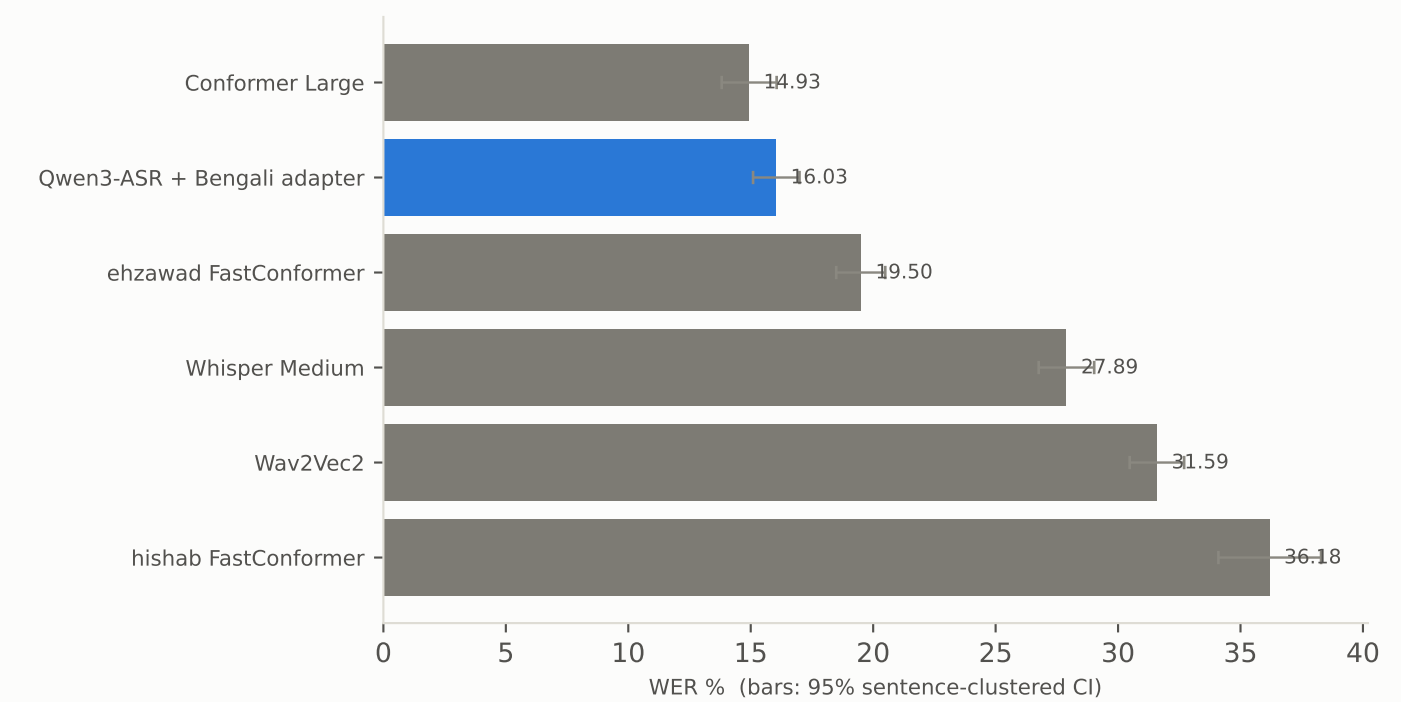
Reading this table

Corpus WER and CER over 1322 utterances, computed from raw hypotheses with the normaliser the earlier report used. Intervals are 95% percentile bootstrap resampling the 499 DISTINCT sentences, not the 1322 recordings: several speakers read the same sentence, so recordings are not independent and a per-utterance interval is optimistic. Both are in the summary.

Accuracy spans 2.4× across these six systems and speed spans 150×. The two orderings are not the same ordering, which is the point of measuring both.

What this report is not
The earlier RTX 5080 report is a different protocol and is left in place unchanged. Its speed numbers were re-measured here because the GPU changed; its end-to-end throughput, queueing latency and failed-request columns are ABSENT here, not re-estimated, because the harness that produced them was never vendored and was unavailable. Speed below is warmed in-process batch-1 inference time, which includes the shared path's WAV staging.

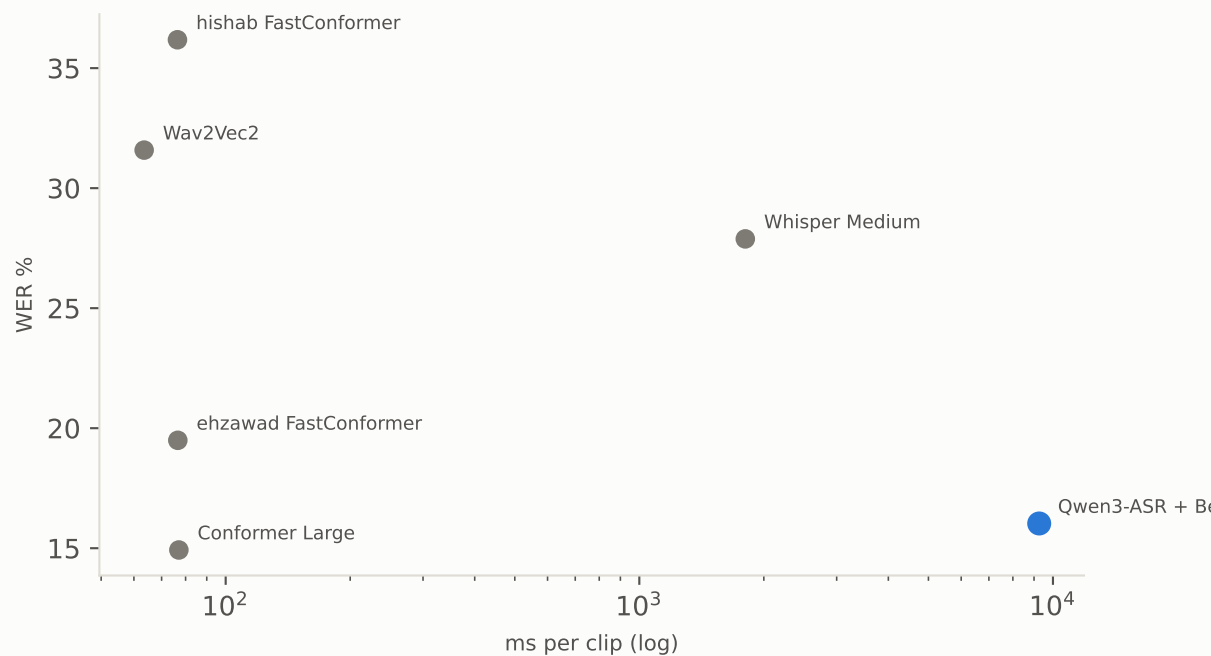
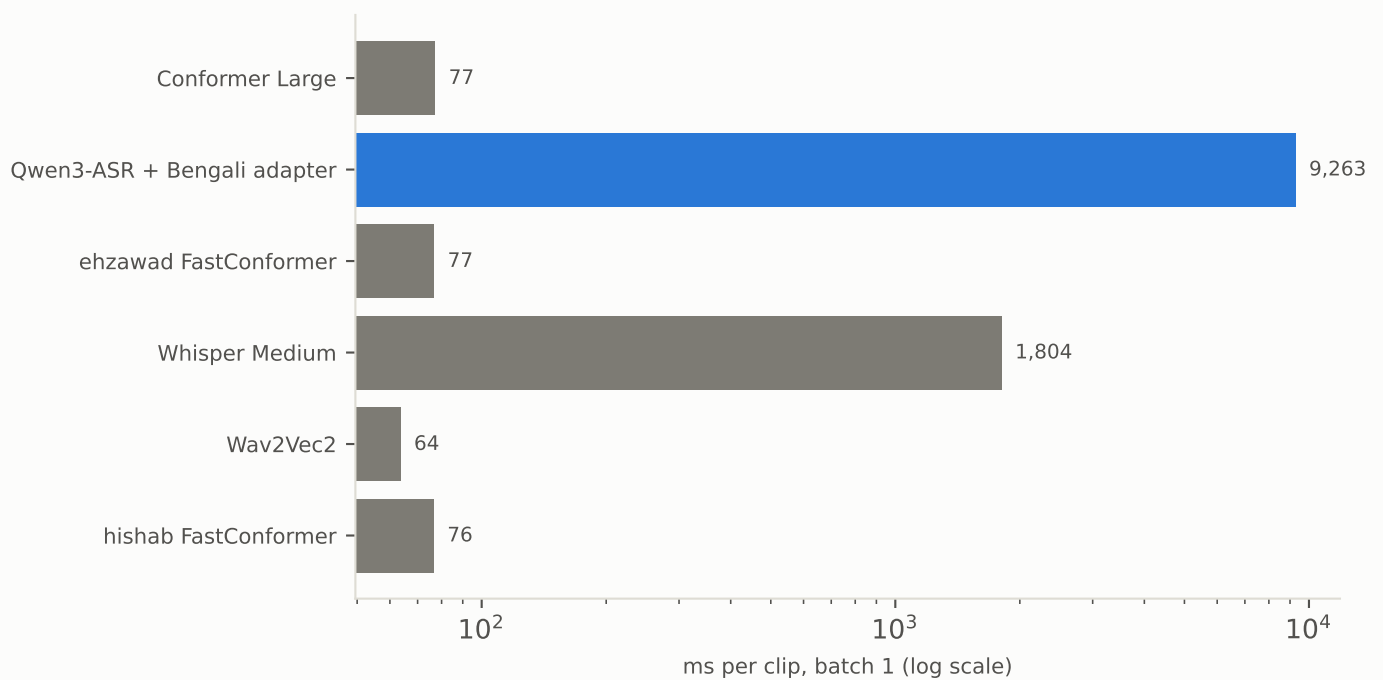
Accuracy



Model	WER test	WER validation	empty hyps	failures
Conformer Large	14.37%	16.23%	0	0
► Qwen3-ASR + Bengali adapter	16.54%	14.83%	0	0
ehzawad FastConformer	19.67%	19.08%	0	0
Whisper Medium	27.88%	27.91%	0	0
Wav2Vec2	31.60%	31.55%	0	0
hishab FastConformer	36.22%	36.09%	0	0

Test and validation are reported separately because validation is normally available during development; the headline merges them only for continuity with the earlier report. No model in this table was trained on either split.

Speed, and what it costs



The adapter is 121× slower per clip than the FastConformer on this GPU: it generates the transcript token by token through a 1.7B decoder, where a CTC model emits an utterance in one forward pass. That is architectural, not a serving artefact. Batch-1 is the interactive-latency workload; batching would help the autoregressive model more than the CTC ones, and is not measured here for any model.

Method, and what it does not establish

One GPU, one model resident at a time. Each checkpoint was loaded alone and released before the next.

One audio path. `asr_core.py` is vendored here and is the only implementation of loading, downmixing, NaN scrubbing, resampling, 25-second segmentation and PCM16 staging. Every model receives the files it stages. Because the adapter needs a different transformers version than the NeMo checkpoints, it runs in a second interpreter — so the staging was executed under both and the SHA-256 of every staged chunk compared, including clips long enough to be segmented. The hashes match, so the bytes reaching each encoder are the same bytes.

Greedy everywhere, batch 1, no external language model or rescoring. Whisper's language is forced to Bengali. A clip that raises becomes an empty hypothesis and stays in the denominator.

Checkpoints are pinned to commit hashes, recorded in the summary.

Limitations

FLEURS is short read speech; it says nothing about conversational, noisy, dialectal or long-form audio, and it is close to Whisper's least favourable case. One run per model, so speed carries no interval. The speed figure is warmed in-process batch-1 time including WAV staging — not pure model compute, and not a serving benchmark. Parameter counts describe the inference system: the adapter row is a 2.04B-parameter model of which 38M were trained. Runtime versions differ between the two interpreters, so each row measures a model-plus-runtime configuration rather than an architecture in isolation.

Absent by necessity

The original harness that produced throughput, queueing latency and failed-request counts was never vendored and was unavailable; those columns are absent

Why the adapter was slow

The 120x gap in the table above is not the audio encoder, and not GPU arithmetic.

Where a clip's time goes

component	value
Autoregressive decode loop	99.7% of the clip
Audio tower (encoder)	16.4 ms
Generated tokens per clip	129.8
Measured cost per decode token	66.05 ms
Memory-bandwidth floor at 768 GB/s	5.34 ms

Per-token cost stayed flat as context grew (67.6 / 66.4 / 66.2 ms at 90 / 180 / 113 generated tokens), so the KV cache was working -- a broken cache grows with position. Prefill likewise did not scale with prompt length. Both point at a fixed per-forward-pass cost. Profiling 12 decode steps found it: 19,225 kernel launches, 1,602 per generated token, with the GPU busy only 42% of the time. RMSNorm ran as four separate kernels and the dynamic cache reallocated K and V every layer, every step. The card was idle, waiting for Python to issue work.

Equal treatment, all 1,322 clips

model	default	+ static cache	speedup	WER after
► Qwen3-ASR + Bengali adapter	9,263.1 ms	1,461.0 ms	6.34x	16.08%
Whisper Medium	1,804.0 ms	467.6 ms	3.86x	27.89%

Merging the 196 LoRA modules into the base weights gave 1.94x; a static KV cache gave a further 4.1x. Accuracy was not assumed to survive that: changing the cache changes bf16 reduction order, so all 1,322 clips were rescored. WER moved +0.051 pp, 95% CI [-0.035, +0.141] -- an interval containing zero, with 1222/1322 transcripts unchanged.

Whisper Medium remains 3.12x faster under identical treatment. Optimising one model and not the other would have reversed the ranking: at 1,461 ms against Whisper's unoptimised 1,804 ms the adapter appears to win, and that would have been an artefact of unequal effort, not a property of the models. The residual gap is structural -- a 2.0B model emitting ~130 tokens per clip against a 769M model emitting ~90 -- and no runtime setting removes it.

A claim that did not survive testing

The adapter's CER (4.475%) is nominally below the best model in the table (4.555%). Paired over identical utterances and clustered by the 499 distinct reference sentences, the difference is -0.081 pp with a 95% CI of [-0.590, +0.414] -- it contains zero. "Lowest CER of the six" is not a claim this benchmark supports; "indistinguishable from the best model on CER" is.